

EXAM NOTES

P2: Exec, Process Upgrade & Low-Level I/O

Beginner-Friendly Reference for Code Output Questions

⚡ FAST REFERENCE — exec() Variants

Function	How arguments are passed	How executable is found
<code>execl(path, arg0, ...)</code>	Listed one-by-one (l = list)	Full path required
<code>execlp(file, arg0, ...)</code>	Listed one-by-one	Searches PATH
<code>execv(path, argv[])</code>	Array (v = vector)	Full path required
<code>execvp(file, argv[])</code>	Array	Searches PATH

📌 TIP

ALL exec variants: if successful → does NOT return. Process is REPLACED.
If exec returns at all → it FAILED. Always check for failure after exec.

1. THE FORK PITFALL — `exit()` IN THE CHILD

This is the #1 mistake beginners make. Look at these two versions of the same code:

✗BUGGY Version — Missing `exit()` in child

```
pid_t pid = fork(); ◀ creates child
if (pid < 0) {
    perror("fork()"); exit(1);
} else if (pid == 0) {
    child_tasks(); ◀ child does its work...
    // NO exit() here ← BUG!
} else {
    parent_tasks();
}
more_parent_tasks(); ◀ ← CHILD runs this too! Bug!
```

❏ WARN

After `child_tasks()`, the child falls through to `more_parent_tasks()`. The child is now running code meant only for the parent! Both parent AND child call `more_parent_tasks()` — double execution, corrupted output.

✓CORRECT Version — `exit(0)` after child work

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork()"); exit(1);
} else if (pid == 0) {
    child_tasks();
    exit(0); // child exits here ← CORRECT ◀ child stops here
} else {
    parent_tasks();
}
more_parent_tasks(); ◀ ← only parent reaches this now
```

❏ KEY

`exit(0)` means 'I finished successfully'. `exit(1)` means 'I failed'. Without `exit()` in the child branch, child will continue executing parent code. RULE: Every child branch MUST end with `exit()` or `return`.

► What to look for in MCQ code output questions

- Is there an `exit()` at the end of the child (`pid==0`) block?
- If NO → child also runs the code after the if-else → look for double output
- If YES → only parent runs the code after the if-else

❏ EXAM

Q: 'How many times is X printed?' — count whether child escapes the if-else.
Q: 'What is wrong with this code?' — look for missing `exit()` in the child branch.

2. PROCESS UPGRADE — WHAT `exec()` DOES

`exec()` replaces the current process with a completely different program.

Think of it like gutting a house and rebuilding the interior — same house number (PID), totally new contents.

The `exec()` Mental Model

Stage	What happens
Before <code>exec()</code>	Process is running your program's code in its address space
<code>exec()</code> called	OS loads a NEW executable from disk into this process's address space
After <code>exec()</code> succeeds	The NEW program starts from its own <code>main()</code> — old code is GONE
After <code>exec()</code> fails	Returns -1; old code continues (<code>exec</code> didn't touch anything)

⚠ WARN

`exec()` does NOT create a new process. The PID stays the same.
`exec()` does NOT return on success. If the line after `exec()` runs → `exec` FAILED.
Open files (including `stdin`, `stdout`, `stderr`) survive `exec()` — very important!

Files Survive `exec()` — Critical Detail

- File descriptors 0 (`stdin`), 1 (`stdout`), 2 (`stderr`) stay open after `exec`
- Any other files opened before `exec` also remain open
- This is how a shell redirects I/O: it sets up FDs then `exec`s the program

```
pid_t pid = fork();
if (pid == 0) {
    // child has FDs 0, 1, 2 open (inherited from parent via fork)
    execlp("gcc", "gcc", "a.c", NULL); ◀ gcc inherits FD 0,1,2
    // if we reach here, execlp() FAILED
    return 1; // always terminate child on exec failure ◀ must exit!
}
```

💡 TIP

The new program (`gcc` above) can write to `stdout`/`stderr` because it inherited FD 1 and 2.
This is why programs you `exec()` can print to your terminal.

3. execl() AND execlp() — ARGUMENT LIST STYLE

Signature

```
#include <unistd.h>

int execl(const char *path, const char *arg0, ..., (char *)NULL);
//      ^path to binary      ^argv[0]                ^MUST end with NULL

int execlp(const char *file, const char *arg0, ..., (char *)NULL);
//      ^filename only      ^argv[0]                ^NULL required
```

Argument Rules

- **arg0 is argv[0] in the new program — by convention this is the program name itself**
- Then come the actual arguments (arg1, arg2, ...) that become argv[1], argv[2], ...
- **MUST end with (char *)NULL — this tells exec() where the argument list ends**
- execl uses a full path; execlp searches the PATH environment variable

❏ WARN

Forgetting the NULL at the end is undefined behaviour — the program may crash or do unexpected things.
The NULL must be cast: (char *)NULL — not just 0 or NULL alone.

Worked Example — adder program

Suppose we have an adder executable that sums its integer arguments:

```
// adder.c - sums all its arguments
int main(int argc, char *argv[]) {
    int sum = 0;
    for (int i = 1; i < argc; i++) // argv[0] = "./adder", skip it
        sum += atoi(argv[i]);
    printf("sum is: %d\n", sum);
    return 0;
}
```

The parent forks, then the child replaces itself with adder:

```
pid_t child = fork();
if (child == 0) {
    printf("In child!\n"); // ◀ prints BEFORE exec
    execl("./adder", "./adder", "1", "2", "3", "10", NULL); // ◀ child becomes adder
    printf("Oops!\n"); // only runs if execl FAILED ◀ exec failed indicator
    return -1;
} else {
    printf("In parent!\n"); // ◀ parent prints this
    execl("expr", "expr", "100", "+", "300", NULL); // ◀ parent becomes expr
    printf("Oops!\n");
    return -1;
}
```

❏ KEY

Expected output (order may vary):
'In child!' then adder prints 'sum is: 16'

'In parent!' then expr prints '400'
Note: 'Oops!' lines only print if exec FAILS — they are error indicators.

How to trace exec() output in exam questions

- Step 1: Identify what program exec() loads (first argument = path/file)
- Step 2: Map the remaining arguments to argv[0], argv[1], argv[2], ... of the new program
- Step 3: The new program runs from its main() with those arguments
- Step 4: Everything printed BEFORE exec() still printed; exec() swaps code forward

□ EXAM

In the adder example above, execl('./adder', './adder', '1', '2', '3', '10', NULL)
→ argv[0]='./adder', argv[1]='1', argv[2]='2', argv[3]='3', argv[4]='10'
→ adder loops i=1..4, sums 1+2+3+10=16, prints 'sum is: 16'

4. execv() AND execvp() — ARRAY STYLE

Same as execl/execlp but arguments are passed as an array instead of a list. Use when you don't know the number of arguments at compile time.

Signature

```
#include <unistd.h>

int execv(const char *path, char *const argv[]);
//      ^full path      ^NULL-terminated array

int execvp(const char *file, char *const argv[]);
//      ^searches PATH  ^NULL-terminated array
```

Building the argv array

```
char *args[] = {"gcc", "-o", "ab", "ab.c", NULL}; ◀ args[4] must be NULL
//      argv[0]="gcc" argv[1]="-o" argv[2]="ab" argv[3]="ab.c"

execvp("gcc", args); ◀ searches PATH for gcc
// equivalent to typing: gcc -o ab ab.c on the command line
```

❏ WARN

The last element of the array MUST be NULL.
Without NULL, execv reads past the array into garbage memory → crash or undefined behaviour.

execl vs execv — when to use which

Situation	Use
Fixed number of args known at compile time	execl / execlp (simpler syntax)
Variable or unknown number of args at runtime	execv / execvp (build array dynamically)
Have the full path to the binary	execl or execv (no PATH search)
Want OS to search PATH for you	execlp or execvp

The exec() Question Pattern — What Might Go Wrong?

From the slides, this code has a problem:

```
pid_t pid = fork();
if (pid < 0) { perror("fork()"); exit(1); }
else if (pid == 0) {
    execlp("genie", "genie", "clean the house", NULL); ◀ what if 'genie' doesn't exist?
}
online_shopping(); ◀ ← parent runs this
```

❏ WARN

Problem: if execlp() fails (program 'genie' not found), the child continues silently.

The child then falls through and may run `online_shopping()` — or worse, garbage code.

Fix: always add error handling + `exit()` after `exec`:

```
execlp("genie", "genie", "clean the house", NULL);  
// If we reach here, execlp failed!  
perror("genie");    // print error message  
exit(1);            // terminate child
```

5. LOW-LEVEL FILE I/O — FILE DESCRIPTORS

Two Levels of File APIs

Level	API style	Type	Features
High-level (C stdlib)	fopen, fclose, fread, fprintf...	FILE *	Buffered, formatted I/O, portable
Low-level (Unix)	open, close, read, write...	int (fd)	Raw bytes, no buffering, system calls

❏ TIP

File Descriptor (fd) is just a small non-negative integer (0, 1, 2, 3, ...). The OS maps this integer to the actual file internally. Always check return value: -1 means error.

The 3 Standard File Descriptors

FD number	Name	FILE* equivalent	Default connection
0	stdin	stdin	Keyboard input
1	stdout	stdout	Terminal output
2	stderr	stderr	Terminal error output

❏ KEY

These 3 FDs are always open when your program starts. They survive `fork()` (child inherits them) and `exec()` (new program keeps them). `fileno(FILE*)` converts a FILE* to its fd: `fileno(stdout) == 1`

Required Headers

```
#include <sys/types.h>    // ssize_t, off_t
#include <sys/stat.h>      // S_IRUSR, S_IWUSR (permission bits)
#include <fcntl.h>         // open(), O_RDONLY, O_WRONLY, etc.
#include <unistd.h>        // read(), write(), close(), lseek()
```


6. open() — OPENING AND CREATING FILES

Signature

```
#include <fcntl.h>

int open(const char *path, int oflag);           // open existing
int open(const char *path, int oflag, int mode); // create new file

// Returns: file descriptor (>= 0) on success, -1 on error
```

oflag — Access Mode (MUST pick exactly one)

Flag	Meaning
O_RDONLY	Open for reading ONLY
O_WRONLY	Open for writing ONLY
O_RDWR	Open for reading AND writing

oflag — Optional Flags (combine with |)

Optional Flag	Meaning
O_CREAT	Create the file if it doesn't exist (requires mode argument)
O_TRUNC	If file exists and is opened for write, erase its contents first
O_APPEND	Always write at the end of the file (don't overwrite)
O_EXCL	Fail if file already exists (use with O_CREAT for atomic create)

Examples

```
int fd;

fd = open("a.txt", O_RDONLY);  ◀ read-only, file must exist

fd = open("a.txt", O_RDWR);    ◀ read+write, file must exist

fd = open("a.txt", O_RDWR | O_TRUNC);  ◀ open + erase existing contents

fd = open("b.txt", O_WRONLY | O_TRUNC | O_CREAT, 0600);  ◀ create or overwrite
// 0600 = owner can read+write; others cannot (octal!)
```

The mode / permissions argument

Only needed when O_CREAT is used. Written in OCTAL (leading 0 is important!).

mode value	Who can do what
0600	Owner: read+write. Group: none. Others: none.
0644	Owner: read+write. Group: read. Others: read.

0755	Owner: read+write+execute. Group: read+execute. Others: read+execute.
0400	Owner: read only. Nobody else can do anything.

❏ **WARN**

0600 is an OCTAL number (base 8), not decimal 600!
The leading 0 tells C the literal is octal. Omitting 0 gives wrong permissions.
Digits: [owner][group][others] — each is a 3-bit combo: 4=read, 2=write, 1=execute

Always check the return value of open()

```
int fd = open("data.txt", O_RDONLY);
if (fd == -1) {
    perror("open");    // prints: open: No such file or directory
    exit(1);
}
// now fd is a valid file descriptor (e.g., 3)
```

7. read(), write(), close(), lseek()

close()

```
int close(int fd);  
// Returns 0 on success, -1 on error  
  
close(fd);    // release the file descriptor  
// Always close fds when done - OS has a limit on open fds per process
```

read()

```
ssize_t read(int fd, void *buf, size_t nbyte);  
  
// fd    = file descriptor to read from  
// buf    = pointer to buffer where data is stored  
// nbyte  = max number of bytes to read  
// Returns: number of bytes actually read, 0 = EOF, -1 = error
```

```
char buffer[100];  
ssize_t n = read(fd, buffer, sizeof(buffer) - 1); ◀ read up to 99 bytes  
if (n == -1) { perror("read"); exit(1); }  
if (n == 0) { /* EOF reached */ } ◀ end of file  
buffer[n] = '\0';    // null-terminate so we can use it as a string  
printf("%s", buffer);
```

❏ WARN

read() may read FEWER bytes than you asked for — always check the return value.
read() does NOT add a null terminator — you must do `buffer[n]='\0'` yourself.
Reading from fd 0 reads from stdin (keyboard or pipe).

write()

```
ssize_t write(int fd, const void *buf, size_t nbyte);  
  
// fd    = file descriptor to write to  
// buf    = pointer to data to write  
// nbyte  = number of bytes to write  
// Returns: number of bytes actually written, -1 = error
```

```
write(1, "Hello\n", 6); ◀ write to stdout (fd=1)  
write(2, "Error!\n", 7); ◀ write to stderr (fd=2)  
  
// Write a string to a file:  
const char *msg = "data to save";  
write(fd, msg, strlen(msg)); ◀ no null-terminator written
```

lseek() — Moving the File Position

```
off_t lseek(int fd, off_t offset, int whence);
```

```
// whence options:  
// SEEK_SET - move to exactly 'offset' bytes from the START  
// SEEK_CUR - move 'offset' bytes from the CURRENT position  
// SEEK_END - move 'offset' bytes from the END of file  
  
lseek(fd, 0, SEEK_SET);    // rewind to beginning  
lseek(fd, 0, SEEK_END);    // jump to end  
lseek(fd, -10, SEEK_END);  // 10 bytes before end  
off_t pos = lseek(fd, 0, SEEK_CUR); // get current position
```

❏ **TIP**

`lseek()` returns the new file position as an `off_t`.
Use `lseek(fd, 0, SEEK_CUR)` to find out where you currently are in the file.
After `fork()`, both parent and child share the same file position — advancing in one advances in both.

8. FILE DESCRIPTORS AFTER fork() AND exec()

After fork()

- **Child gets copies of all parent's file descriptors**
- Both parent and child refer to the SAME underlying file description (shared position)
- Reading in child advances position → parent sees the advanced position too
- Closing fd in child does NOT close it in parent (and vice versa)

```
int fd = open("file.txt", O_RDONLY);
fork();
// Now BOTH parent and child have fd open
// If child reads 5 bytes, the shared position advances 5 bytes
// Parent's next read starts 5 bytes further in
```

⚠ WARN

Shared file position after fork() is a common exam trap.
If both parent and child read from the same fd, they interleave reads — not each read from start.

After exec()

- **File descriptors remain open — exec does NOT close them**
- The new program inherits FDs 0, 1, 2 (stdin, stdout, stderr)
- Any other open FDs are also inherited unless marked O_CLOEXEC
- This is intentional — allows shell to set up redirections before exec

⚡ KEY

fork() + exec() pattern: fork creates child, child sets up FDs (redirects), then exec's program.
The exec'd program sees the redirected stdin/stdout because FDs persist through exec.

Full Picture

Operation	Effect on File Descriptors
<code>fork()</code>	Child gets copies of all parent FDs; shared underlying file position
<code>exec()</code>	All FDs stay open; new program inherits them
<code>close(fd)</code>	Only affects the calling process; other process still has it open
<code>exit()</code>	All FDs are closed (OS cleans up)

9. HOW TO TRACE CODE OUTPUT — STEP BY STEP

Most exam questions show a fork/exec program and ask what it prints. Follow this process:

The 6-Step Tracing Method

Step	What to do
1. Find <code>fork()</code>	Mark the line — everything before runs once in ONE process
2. Split into parent/child	Draw two columns: Parent (<code>pid>0</code>) Child (<code>pid==0</code>)
3. Trace each branch	Follow the code in each column independently
4. Check for <code>exit()</code>	Does the child exit before falling through? If no → child runs extra code
5. Check for <code>exec()</code>	If exec succeeds → process runs new program. Trace new program's output
6. Check <code>wait()</code>	Does parent wait? If yes → parent blocks until child finishes, then continues

Worked Trace — Fork + Exec Example

Trace this code step by step:

```
int main() {
    pid_t child = fork();          // SPLIT HERE ◀ creates two processes
    if (child == 0) {
        printf("In child!\n");    // child prints this ◀ CHILD output 1
        execl("./adder", "./adder", "1", "2", "3", "10", NULL); ◀ child BECOMES
adder
        printf("Oops!\n");        // only if execl failed
        return -1;
    } else {
        printf("In parent!\n");   // parent prints this ◀ PARENT output 1
        execl("expr", "expr", "100", "+", "300", NULL); ◀ parent BECOMES expr
        printf("Oops!\n");
        return -1;
    }
}
```

□ KEY

CHILD trace: prints 'In child!' → exec succeeds → adder runs with args 1 2 3 10 → prints 'sum is: 16'
PARENT trace: prints 'In parent!' → exec succeeds → expr runs: 100+300 → prints '400'
'Oops!' lines do NOT print (exec succeeded).
Order of 'In child!' vs 'In parent!' is non-deterministic — either can come first.

Tracing Multiple `fork()` Calls

```
fork();          // Process count: 1 → 2 ◀ now 2 processes
```

```
fork();           // Each of 2 forks → 2 → 4 ◀ now 4 processes
printf("hi\n");   // Each of 4 processes prints ◀ prints 4 times
```

❑ TIP

Without `exit()` in between, each `fork()` doubles the count. N forks → 2^N processes. Each of those 2^N processes executes all code after the last `fork()`.

Tracing `fork()` in a Loop — The Classic Trap

```
for (int i = 0; i < 3; i++) {
    pid_t pid = fork(); ◀ forks inside loop!
    if (pid == 0) {
        printf("child %d\n", i);
        exit(0); // ESSENTIAL — stop child from looping ◀ child exits here
    }
    // parent continues loop
}
// parent calls wait 3 times
for (int i = 0; i < 3; i++) wait(NULL);
```

❑ EXAM

With `exit(0)` in child: parent forks 3 children, each prints once → 3 total prints
WITHOUT `exit(0)`: child 0 continues loop, forks more children → exponential! → fork bomb

10. MCQ QUICK-ANSWER TABLE

Question	Answer
<code>execl()</code> first argument is...?	Path to the executable file
<code>execl()</code> second argument (<code>arg0</code>) is...?	Conventionally the name of the program (<code>argv[0]</code>)
What terminates an <code>exec()</code> argument list?	<code>(char *)NULL</code> — the NULL pointer, cast to <code>char*</code>
What does <code>exec()</code> return on SUCCESS?	Nothing — it does NOT return at all
What does <code>exec()</code> return on FAILURE?	-1 (the line after <code>exec()</code> runs only on failure)
Do open files survive <code>exec()</code> ?	YES — all FDs including <code>stdin/stdout/stderr</code> stay open
Do open files survive <code>fork()</code> ?	YES — child inherits copies; file position is SHARED
<code>execvp</code> vs <code>execl</code> : difference?	<code>execvp</code> searches PATH; <code>execl</code> needs the full path
<code>execv</code> vs <code>execl</code> : difference?	<code>execv</code> takes an array of args; <code>execl</code> takes a list
What is fd 0 / 1 / 2?	0= <code>stdin</code> , 1= <code>stdout</code> , 2= <code>stderr</code>
<code>open()</code> returns -1 means...?	Error — file not found, no permission, etc.
<code>O_TRUNC</code> flag does what?	Erases existing file contents when opening for write
<code>O_CREAT</code> requires what extra arg?	mode (permissions) — e.g., 0600
Why does mode need a leading 0?	Leading 0 = octal literal in C — permissions are octal
<code>read()</code> returns 0 means?	EOF — end of file reached, no more data
Does <code>read()</code> add null terminator?	NO — you must do <code>buf[n]='\0'</code> yourself
<code>SEEK_SET</code> / <code>SEEK_CUR</code> / <code>SEEK_END</code> ?	From start / from current pos / from end of file
Missing <code>exit()</code> in child branch?	Child falls through and runs parent's code too — BUG
Line after <code>exec()</code> runs — what happened?	<code>exec()</code> FAILED (program not found or permission denied)
<code>fileno(stdout)</code> returns?	1 (the file descriptor for <code>stdout</code>)

11. FINAL CHEATSHEET — ALL FUNCTIONS AT A GLANCE

Function / Concept	Quick fact
<code>fork()</code>	Clone process. Returns child PID to parent, 0 to child, -1 on error
<code>exit(code)</code>	Terminate this process with exit code
<code>wait(NULL)</code>	Block until any child dies; reap zombie
<code>execl(path,a0,...,NULL)</code>	Replace process with program at path; list args; end with NULL
<code>execvp(file,a0,...,NULL)</code>	Same as <code>execl</code> but searches PATH for file
<code>execv(path, argv[])</code>	Replace process; args in NULL-terminated array
<code>execvp(file, argv[])</code>	Same as <code>execv</code> but searches PATH
<code>exec()</code> on success	Does NOT return — old code gone, new <code>main()</code> starts
<code>exec()</code> on failure	Returns -1; old code continues — must handle and <code>exit()</code> !
<code>open(path, flags)</code>	Open file; returns fd or -1
<code>open(path, flags, mode)</code>	Open/create file with permissions; mode in octal
<code>close(fd)</code>	Release file descriptor
<code>read(fd, buf, n)</code>	Read up to n bytes; returns bytes read, 0=EOF, -1=error
<code>write(fd, buf, n)</code>	Write n bytes; returns bytes written, -1=error
<code>lseek(fd, offset, SEEK_SET)</code>	Move to absolute position
<code>lseek(fd, 0, SEEK_END)</code>	Jump to end of file
FD 0 / 1 / 2	stdin / stdout / stderr
<code>fileno(fp)</code>	Get fd from FILE*
<code>O_RDONLY/O_WRONLY/O_RDWR</code>	Read-only / write-only / read+write
<code>O_CREAT O_TRUNC O_APPEND</code>	Create / truncate / append
FDs after <code>fork()</code>	Inherited; shared file position
FDs after <code>exec()</code>	Survive <code>exec</code> ; new program sees same FDs
Missing <code>exit()</code> in child	Child runs parent code → double execution bug

Good luck! You've got this. ☐